

# CCTA Unit - IV

## Unit - IV - Introduction to Multi-Cloud and Enterprise Cloud Architecture

### Syllabus:

#### Part - A: Introduction to Multi-Cloud

1. Introduction to Multi-Cloud
2. Setting Out a Real Strategy for Multi-Cloud
3. Analyzing the Enterprise Strategy for the Cloud
4. Introducing the Scaffold for Multi-Cloud Environments
5. Understanding Identities and Roles in the Cloud

#### Part - B: Enterprise Cloud Architecture

1. Defining Architecture Principles for Multi-Cloud
2. Using Quality Attributes in Architectures
3. Defining Principles from use-cases
  - a. Business Principles
  - b. Principles for Security and Compliance
  - c. Data Principles
  - d. Application Principles
  - e. Infrastructure and Technology Principles
  - f. Principles for Processes

## 1 Introduction to Multi-Cloud

In today's cloud-centric world, most companies are already using multiple cloud services—often unknowingly. This strategy, known as **multi-cloud**,

involves utilizing services from different cloud providers to meet various business needs. It's a growing trend driven by the need for agility, flexibility, and the ability to choose the best services for specific functions.

## 1.1 Understanding Multi-Cloud Concepts

- Multi-cloud deployments often result in:
  - Uncontrolled growth of operational **costs**
  - Lack of **visibility** into system ownership and management
  - Increased **security risks** due to system sprawl
- Establishing a clear definition and understanding of **multi-cloud** is essential before implementing a solution.

### 1.1.1 Definition of Multi-Cloud

- Multi-cloud involves the simultaneous use of **two or more cloud computing platforms**
- Deployments may include:
  - **Public clouds**
  - **Private clouds**
  - Or a combination of both
- Primary goals of multi-cloud:
  - Ensuring **redundancy** in case of hardware or software failures
  - Avoiding **vendor lock-in** and ensuring service flexibility

### 1.1.2 Origin of Existing IT Deployments

- Most enterprise environments originate from **traditional datacenters**
- Traditional IT includes:
  - **Physical infrastructure**
  - **Virtualized systems**
  - Hosting a range of **business functions and applications**
- Current state-of-the-art technology evolves into **legacy infrastructure** over time

- Traditional IT commonly refers to systems hosted in **privately owned, on-premise datacenters**

### 1.1.3 Understanding the Role of Private Cloud

- Many organizations describe their virtualized environments as **private clouds**
- These environments may reside in:
  - **External datacenters**
  - **Self-owned, on-premise infrastructure**
- Typical features of private clouds:
  - **Centrally managed** platforms
  - **Consumption-based billing** to internal business units
- Broad interpretations of private cloud exist, often based on internal IT service delivery models

### 1.1.4 Public Cloud Characteristics

- Public cloud platforms include:
  - **Amazon Web Services (AWS)**
  - **Microsoft Azure**
  - **Google Cloud Platform (GCP)**
- Key characteristics of public cloud:
  - Offers **on-demand IT services** such as compute, storage, and networking
  - Accessed via the **public internet**
  - **Cloud provider** maintains and manages the physical infrastructure
  - Organizations **rent services**, eliminating the need for datacenter investments

### 1.1.5 Alternative Perspective on Multi-Cloud

- Multi-cloud can also represent a **best-of-breed strategy**
- Combines cloud services and tools from different platforms to:

- Maximize **business value**
- Enhance **service compatibility**
- Increase **deployment flexibility**
- Multi-cloud may consist of:
  - Services exclusively from different **public cloud platforms**
  - A **hybrid combination** of public and private clouds
- True multi-cloud strategy requires more than service aggregation; it demands **coordination, customization, and integration**

## 1.1.6 Smart Phone Analogy to explain Multi-Cloud

### Connectivity Requirement:

- A new smartphone without network connectivity offers limited functionality
- Initial setup requires:
  - **Mobile network subscription**
  - Or **Wi-Fi access**

### Application Enablement:

- Applications are downloaded from online catalogs such as **app stores**
- Applications originate from:
  - Various **organizations** (e.g., banks, retailers)
  - Written in **different programming languages**
  - Compatible with **different operating systems** (e.g., iOS, Android)

### Personal Configuration:

- Applications are **configured** based on user preferences
- Configuration enables personalized **service delivery** and **data access**

### System Updates:

- Operating systems receive **automatic updates**
- Installed applications generally continue functioning post-update
- Stored or cloud-accessed data remains **intact and available**

### Integrated Service Platform:

- Smartphone functions as a **platform for personalized applications and data**
- Underlying complexity is **abstracted**, offering a **transparent user experience**

### Mapping the Analogy to Cloud:

- A cloud platform resembles a smartphone platform:
  - Requires **initial configuration** and **network connectivity**
  - Functions as an **integration point** for applications and data
- Cloud portal provides:
  - A platform connected via the internet
  - An environment where applications and data can be uploaded
  - Configurable **access controls** for authorized users
- Multi-cloud strategy enables:
  - Application and data hosting across **multiple platforms**
  - **Migration** of applications between platforms



Figure - 1 - Analogy of the smartphone - a true multi-cloud concept

### Real World Comparison:

- Example: Facebook application functions identically on both Android and iOS
  - Same **account**
  - Same **data**
  - Different **underlying platforms**
- Multi-cloud delivers similar outcomes:
  - **Cross-platform application portability**
  - **Consistent access to data and services**
  - **Seamless user experience** across diverse infrastructures

## 1.2 Multi-Cloud - More than just Public and Private

### 1.2.1 Hybrid v/s Multi-Cloud

- **Hybrid** and **multi-cloud** are often used interchangeably but differ in architecture and intent.
- **Hybrid platforms** are typically **homogeneous** (same vendor stack).
  - Example: Azure Public Cloud combined with Azure Stack on-premises.
- **Multi-cloud platforms** are **heterogeneous** (multiple vendor stacks).
  - Example: Use of both Azure and AWS.

### 1.2.2 Core Definitions

- **Hybrid:** Combines **on-premises infrastructure** (often private cloud) with **public cloud**.
- **Multi-cloud:** Uses services from **two or more different cloud providers**.
- **Private Cloud:** Dedicated cloud resources used by a **single organization or user**.
- **Public Cloud:** Shared infrastructure with **isolated tenant environments**; resources such as networking may be shared, but data remains private.

### 1.2.3 Hybrid Cloud as a Common Enterprise Model

- **Hybrid deployment** combines:
  - An on-premises **private cloud** stack
  - A **public cloud** platform
- Common in enterprise environments.
- Industry consensus: **Hybrid cloud** is expected to become the **most widely implemented** cloud deployment model in the near future.

## 1.2.4 Reasons for Adopting Hybrid Models

### 1. Security and Data Sovereignty

- Certain types of **sensitive data** cannot be hosted outside specific geographic or jurisdictional boundaries.
- Example: Data residency regulations in the **European Union (EU)** may restrict access by foreign entities (e.g., US-based companies).
- Even though public clouds offer robust **security frameworks, legal and compliance requirements** may still necessitate **on-premises storage**.

### 2. Latency-Sensitive Applications

- Applications requiring **real-time responsiveness** may not perform optimally in the public cloud.
- Example 1: **Print servers** — Cloud-hosted spoolers may introduce noticeable delays unless carefully configured and placed in nearby regions.
- Example 2: **Retail warehouses** — Barcode/QR scanning systems for inventory control need **low-latency access**, preferably close to the warehouse floor.
- Example 3: **Mission-critical infrastructure** — Systems used in **hospitals** (operating rooms) and **power plants** must maintain **local control** due to latency, reliability, and risk factors.

## 1.2.5 Agility through Public Cloud Development

- **Agile development environments** and **cloud-native apps** are usually first built in the **public cloud** due to:
  - Rapid provisioning

- Access to scalable services
- Integration with DevOps and CI/CD tools
- Developers commonly initiate cloud adoption by launching **new environments and services** in the public cloud.
- Over time, this leads to a **hybrid IT architecture**:
  - **Private datacenter systems**: host **sensitive or latency-critical production systems**
  - **Public cloud**: used for **application development and testing**

## 1.2.6 Overlapping Scenarios

- Certain deployment models involve **private infrastructure inside public cloud environments**:
  - Example: Dedicated private hosts provisioned within Azure or AWS.
- Alternatively, cloud services may run on **private hardware in a local datacenter**:
  - Example: Use of **Azure Stack** or **AWS Outposts**.
- These scenarios may introduce **confusion** between definitions of hybrid and multi-cloud.
  - A **hybrid model** involves **on-premises + public cloud**.
  - A **multi-cloud model** involves **multiple distinct cloud providers**, regardless of on-premises presence.

## 1.3 Introducing the Main Players in the field

### 1.3.1 Defining Public and Private Cloud

- **Public Cloud**:
  - Computing services offered by **third-party providers** over the **public internet**
  - Available to **anyone** who wants to use or purchase them
- **Private Cloud**:
  - Computing services offered over the **internet or internal networks**



- Access limited to **select users**, not the general public

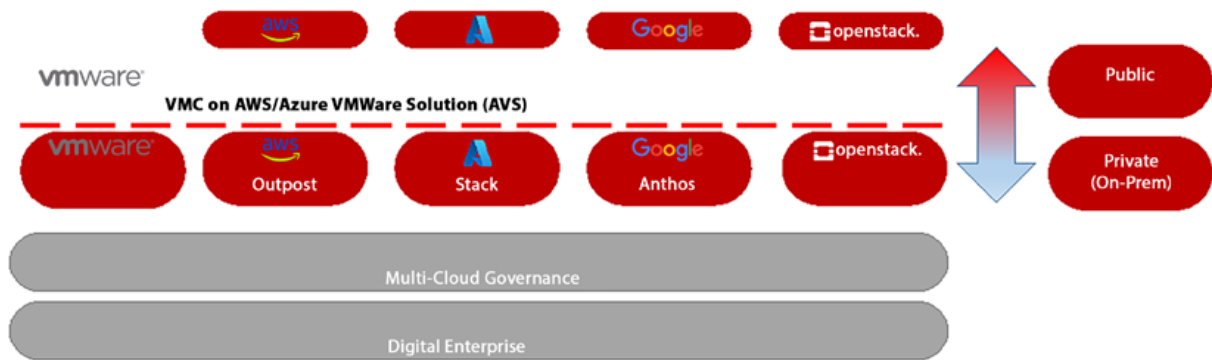


Figure - 2 - An example multi-cloud portfolio: the main players

### 1.3.2 Public Cloud Providers

- Major public cloud providers:
  - **Amazon Web Services (AWS)**
  - **Microsoft Azure**
  - **Google Cloud Platform (GCP)**
  - **Oracle Cloud Infrastructure (OCI)**
  - **Alibaba Cloud**
  - **OpenStack-based public clouds**, e.g., Rackspace
- All major providers offer **managed services** for building cloud environments
  - Conceptual similarities across providers
  - Implementation and technology stacks differ significantly
- **Multi-cloud management complexity** arises from provider-specific implementations

### 1.3.3 Private Cloud Platforms

- Majority of enterprises select from **Azure**, **AWS**, and **GCP** for workload hosting
- Oracle Cloud and Alibaba Cloud gaining **market share** and **strategic importance**
- Increasing regulations related to **compliance**, **security**, and **privacy** drive cautious adoption

- Dominant private cloud technologies:
  - **VMware**: Industry-standard virtualization platform
  - **Microsoft Hyper-V**
  - **OpenStack**: Popular in Europe for data sovereignty

### 1.3.4 Private Cloud Stack Examples

#### VMware:

- Started with **x86 server virtualization**
- Expanded to:
  - **vSAN** for storage virtualization
  - **NSX** for network virtualization and micro-segmentation
- Collaborations:
  - **VMware Cloud on AWS (VMConAWS)**: Seamless private-to-public extension
  - **Azure VMware Solution (AVS)**: Migration of VMware workloads to Azure
- Enhancements:
  - Stronger **containerization capabilities** via Tanzu portfolio
  - Expanded presence in **cloud-native** and **security** domains

#### OpenStack:

- **Open-source IaaS platform**
- Uses **KVM** as the primary hypervisor
- Provides a **vendor-neutral** alternative to hyperscaler lock-in
- Adopted by providers like **IBM (Bluemix)** and **Fujitsu (K5)**
- Challenges:
  - High complexity
  - Limited ecosystem richness compared to hyperscalers
  - **Declining enterprise adoption**, but still relevant in specific sectors

### 1.3.5 Public Cloud Extensions to Private Infrastructure

#### AWS Outposts:

- Preconfigured **on-premise rack appliance**
- Offers:
  - **EC2, EBS, Databases, EKS**
  - Same **APIs** and **controls** as AWS public cloud
- Complements VMConAWS:
  - VMConAWS: **Private cloud → Public cloud**
  - Outposts: **Public cloud → Private infrastructure**
- Appliance options:
  - **Single-server models**
  - **Full rack solutions**
  - Fully managed by **AWS**

#### Google Anthos:

- Not a hardware appliance; runs on top of **virtualized infrastructure**
- Extends **Google Kubernetes Engine (GKE)** to on-premises
- Focused on:
  - **Containerized workloads**
  - **Cloud-native transformation**
- Incorporates:
  - **Istio** (microservices mesh)
  - **Knative** (scalable cloud-native deployment)

#### Azure Stack Portfolio:

- Includes:
  - **Azure Stack HCI**
  - **Azure Stack Hub**
  - **Azure Stack Edge**

#### **Azure Stack HCI:**

- **Standalone box** with compute, storage, and network
- Runs **Hyper-V virtual machines**
- Managed via **Windows Admin Center**
- Can run **disconnected from Azure**
- Connects to Azure for:
  - **Azure Site Recovery**
  - **Azure Backup**
  - **Azure Monitor**
  - **Microsoft Defender**

#### ***Azure Stack Hub:***

- **Integrated extension** of Azure public cloud
- Supports:
  - **VMs**
  - **Apps**
  - **Resource management via Azure Portal or PowerShell**
- Offers:
  - **Fault domain support**
  - **Availability sets**
  - **Consistency with Azure Resource Providers**
- Recommended model:
  - **Development** on public Azure
  - **Production deployment** on Azure Stack Hub (for compliance/local hosting)

#### ***Azure Stack Edge:***

- Designed for **edge data processing**
- Supports:
  - **Containers**
  - **AKS clusters**

- **Azure Machine Learning integration**
- Enables:
  - Local **AI/ML model execution**
  - **Preprocessing** before cloud upload
  - Utilizes **FPGAs and GPUs** for acceleration

#### **Azure Arc:**

- Enables management of **non-Azure resources**:
  - Physical/virtual servers
  - Kubernetes clusters
  - SQL Server instances
- Supports:
  - **Azure data services anywhere**
  - **GitOps-based configuration**
  - **Policy-based governance**
- Connection process:
  - Install **Arc agent**
  - Configure **network settings**
  - Register required **resource providers**
- Manages:
  - **On-premises resources**
  - **Resources in other cloud providers** (e.g., AWS, GCP)

### **1.3.6 Final Observations**

- Cloud landscape includes **multiple providers and platforms**, each with:
  - Unique features
  - Regulatory considerations
  - Suitability for different use cases
- Enterprise strategies often include **workloads across multiple platforms**

- Multi-cloud is driven not just by avoidance of vendor lock-in, but by the realization that **no single cloud fits all use cases**
- Key objective: becoming **cloud-fit**, not cloud-first

### 1.3.7 Emerging Players

#### Market Overview:

- Cloud computing market remains **dominated** by major providers:
  - **Amazon Web Services (AWS)**
  - **Microsoft Azure**
  - **Google Cloud Platform (GCP)**
- **New players** are emerging in public and private cloud segments
- Growth of regional players is driven by:
  - **Geopolitical factors**
  - **Compliance and data sovereignty requirements**
  - **Industry-specific regulations**

#### Geographical Influence on Cloud Adoption:

- In certain countries or industries, **American cloud providers** may not be allowed
- Some regulations require **local data residency** and **in-country provider presence**
- Compliance rules significantly influence **cloud provider selection**

#### China - Based Providers:

##### ***Alibaba Cloud:***

- One of China's leading cloud providers
- Offers a **wide variety of services**
- Expanding **global footprint**
- Positioned as a **competitor** to major American providers

##### ***Tencent Cloud:***

- Strong presence in the **Chinese market**

- Also globally available, but primarily **focused on domestic demand**

#### **European Initiative: Gaia-X:**

- Gaia-X is a **European cloud initiative**
- Based in the **European Union (EU)**
- Focused primarily on the **healthcare sector**
- Ensures **privacy-sensitive patient data** remains hosted within the EU
- Supports **European sovereignty** in cloud services

#### **System Integrators Entering Cloud Market:**

- Large IT and tech companies have launched cloud offerings to **fill specific niches**

#### **IBM Cloud:**

- Collaborates with **Red Hat**
- Positioned for **hybrid cloud** and **enterprise integration use cases**

#### **Fujitsu (Japan):**

- Previously offered **K5**, a fully **OpenStack-based public cloud**
- **Global ambitions** reduced due to high investment requirements
- Could not match **scale and ecosystem** of hyperscalers like AWS or Azure

#### **Scope and Relevance of Emerging Providers:**

- Emerging providers may offer:
  - **Targeted solutions** for compliance-heavy industries
  - **Niche functionalities** and **regional hosting capabilities**
  - But generally lack the **breadth and scale** of major public cloud platforms

## **1.4 Evaluating Cloud Service Models**

- Early cloud platforms functioned as **shared datacenters**
  - Hosted **multiple customers**
  - Provided shared **compute, network, and storage** resources

- Over time, cloud platforms evolved into a variety of **service models** offering different levels of abstraction and management responsibility

### 1.4.1 Infrastructure as a Service (IaaS)

- Most common entry point for enterprises migrating to cloud
- Involves “**lift and shift**” of virtual machines to cloud-hosted infrastructure
- **Managed by the cloud provider:**
  - **Network**
  - **Storage**
  - **Compute**
  - **Virtualization layer**
- **Managed by the customer:**
  - **Operating systems**
  - **Applications**
  - **Middleware**
  - **Data**
- Underlying infrastructure is **virtualized**, enabling multi-tenant hosting on shared physical servers

### 1.4.2 Platform as a Service (PaaS)

- Offers **greater abstraction** than IaaS
- Cloud provider manages:
  - **Operating systems**
  - **Middleware**
  - **Runtime environments**
- Customer focuses on:
  - **Running applications**
  - **Using platforms such as database services**
- Example: Deploying **PostgreSQL** or **MySQL** without managing the underlying OS or DBMS installation



### 1.4.3 Software as a Service (SaaS)

- Complete software stack managed by the cloud provider
- Includes:
  - **Infrastructure**
  - **Platform**
  - **Application software**
  - **Data management**
  - **Updates and patches**
- Accessed by end users via:
  - **Web portals**
  - **APIs**
  - **Subscription models**
- No need for local installation or infrastructure setup

### 1.4.4 Function as a Service (FaaS)

- **Serverless computing model**
- Developers write **functions**, not full applications
- No need to manage:
  - **Servers**
  - **Provisioning**
  - **Scaling**
- Execution is **event-driven**:
  - Resources (CPU, memory) used **only when needed**
  - Reduces cost by avoiding always-on infrastructure

### 1.4.5 Container as a Service (CaaS)

- Supports running, scaling, and orchestrating **containers**
- Built primarily on **Kubernetes** clusters
- Cloud provider handles:

- **Kubernetes cluster setup**
- **Management and scaling**
- Developers manage:
  - **Container images**
  - **Application-level logic**
- CaaS simplifies:
  - **Container orchestration**
  - **Resource provisioning**
  - **Lifecycle management**

#### 1.4.6 Anything as a Service (XaaS)

- Refers to **generalization** of the “as-a-Service” model
- Common types:
  - **HaaS** – Hardware as a Service
  - **DaaS** – Desktop as a Service
  - **DBaaS** – Database as a Service
- Extends beyond IT:
  - **Food delivery platforms**
  - **Taxi booking apps**
  - **Telemedicine services**
- Uses **cloud as the digital enabler** to deliver traditional and new services as **on-demand digital offerings**

## 2 Setting Out a Real-Strategy for Multi-Cloud

### 2.1 Business Goals Driving Cloud Strategy

- Cloud strategy originates from **business strategy**
- Common business goals:

- Increase **brand awareness**
- Achieve **faster product releases**
- Improve **profit margins**
- Ultimately, **increase revenue**
- Strategy must align with **value creation and growth objectives**

## 2.2 Role of Enterprise Architecture in Strategy Definition

- **Enterprise architecture** connects **business goals** to **IT strategy**
- Common frameworks for enterprise architecture:
  - **TOGAF** (The Open Group Architecture Framework)
  - **IT4IT**
  - **ITIL**
- These frameworks provide **structured approaches** for aligning IT with business needs

## 2.3 TOGAF's ADM Cycle

- **ADM (Architecture Development Method)** is the core of TOGAF
- Follows the **B-D-A-T** cycle:
  - **Business**: Identify business objectives and needs
  - **Data**: Define data supporting those needs
  - **Applications**: Specify how data is processed
  - **Technology**: Determine enabling technologies
- Matches well with **multi-cloud architecture**, where technology must support the business without being the central focus

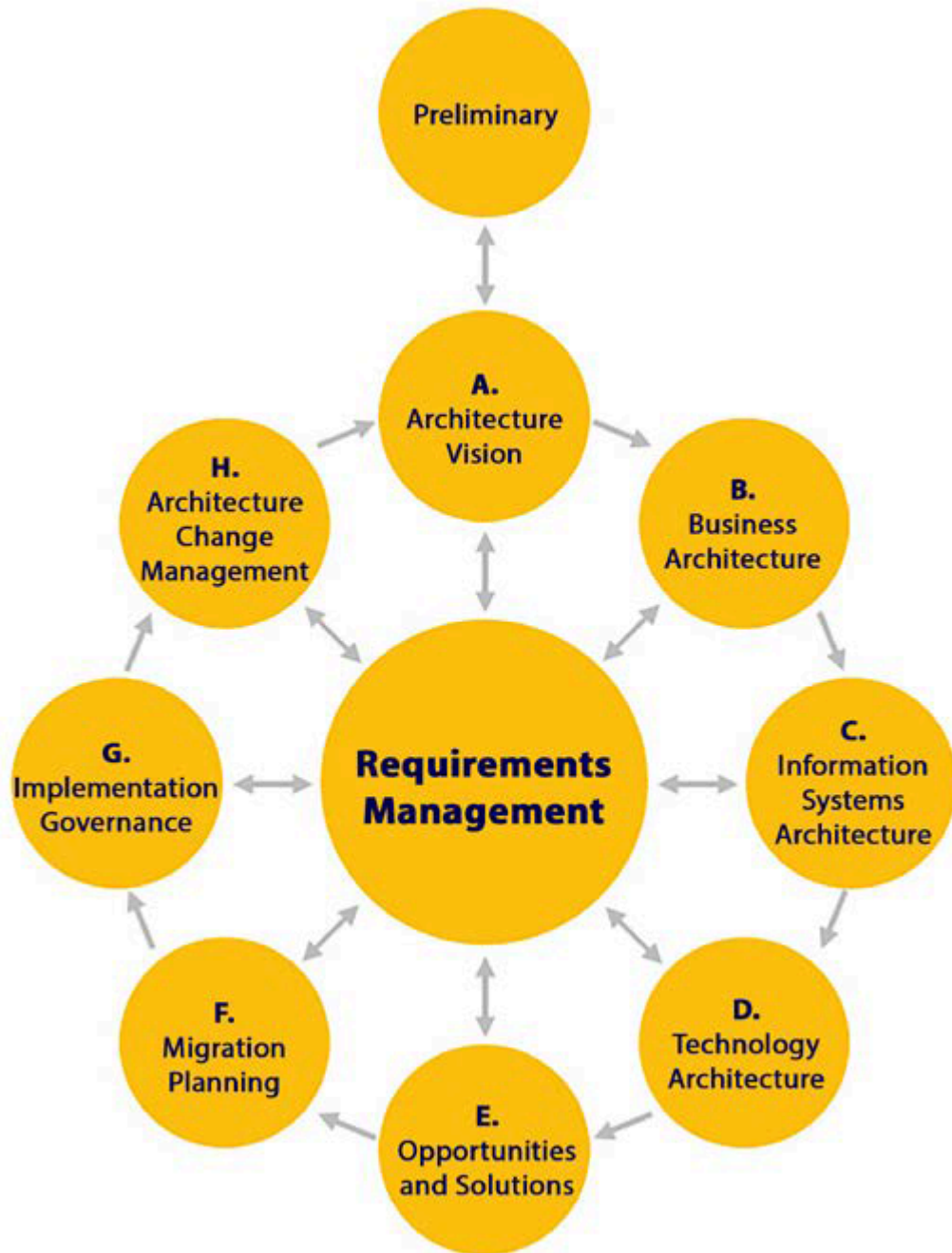


Figure - 3 - The ADM cycle in the TOGAF

## 2.4 Multi-Cloud: Flexibility with Complexity

- Multi-cloud offers:
  - **Freedom of choice**
  - **Technology flexibility**
  - **Avoidance of vendor lock-in**
- Risks associated with multi-cloud:

- **Loss of focus**
- **Increased management complexity**
- **Fragmentation of architecture**
- Cloud strategies are required to **manage complexity and align with transformation goals**

## 2.5 Transformation of Business and IT

- Most organizations are transitioning from **traditional IT environments** to **digital enterprises**
- IT has become a **core business function** across industries
- Historical shift:
  - In the 1990s–early 2000s: widespread **IT outsourcing**
  - Recent decade: organizations **internalize IT** as a **strategic enabler**
- Widely accepted belief:
 

“Every company is a software company”

(as quoted by both **Watts Humphrey** and **Satya Nadella**)

## 2.6 Examples of IT-Centric Business Transformation

- **Banks** as a representative example:
  - Process **massive data streams**
  - Run **data analytics**
  - Develop **digital apps** for customer interaction
  - Require **cloud-native and hybrid cloud platforms**

## 2.7 Best-of-Breed in Multi-Cloud Strategy

- Best-of-breed approach involves selecting **the most suitable service or provider** for each use case
- Typical multi-cloud strategy includes:
  - **Traditional workloads** on VMs
  - Increasing adoption of:

- **PaaS**
- **SaaS**
- **Containers**
- **Serverless functions**
- Move toward **microservices** and **cloud-native** designs

## 2.8 From Cloud-First to Cloud-Fit

- Strategy focus:
  - Not on adopting **cloud for everything**
  - But on adopting the **right cloud** for the **right workload**
- **Cloud-fit** strategy:
  - Optimizes **workload placement**
  - Ensures **alignment with business and operational needs**
  - Avoids unnecessary complexity from adopting **multi-cloud without justification**

# 3 Analyzing the Enterprise Strategy for the Cloud

## 3.1 Understanding Enterprise Strategy

- Goal of any business: **generating revenue**
- **Strategy** is defined by *how* revenue is generated using products and services
- A strong strategy balances:
  - **Timing**
  - **Access to and interpretation of data**
  - **Decisiveness in decision-making**
- Strategy should translate into **specific business use cases**, such as:
  - Delivering products via **new models** (e.g., SaaS)

- Enabling **business continuity** through disaster recovery in the cloud
- Achieving **faster time to market** using cloud-based development environments
- Conducting **big data analytics** using cloud-based data lakes

## 3.2 Drivers of Business Strategy

- Business drivers typically fall into four categories:
  - **Financial objectives**
  - **Customer objectives**
  - **Product objectives**
  - **Internal (operational) objectives**
- Strategy must ensure:
  - Products are priced acceptably to customers
  - Sufficient **profit margins** are maintained
  - **Customer needs** are accurately captured using:
    - **Quality Function Deployment (QFD)**
    - **House of Quality (HOQ)**
    - **Voice of the Customer (VOC)**

## 3.3 Importance of Timing and Agility

- **Timing** is a critical but unpredictable factor in business planning
  - Example: Pandemic of 2020 shifted focus from growth to **cost optimization**
- **Business agility** is essential for adapting to unexpected changes

## 3.4 Business agility defined

- Capability of an enterprise to **respond quickly to market changes**
- Requires transformation at both:
  - **Organizational level**

- **Technology level**

## 3.5 Organizational Changes for Agility

- Adoption of **smaller, agile teams**
  - Teams focus on short-term goals in **sprints**
  - Direct engagement with customers
- Continuous delivery of **incremental value**

## 3.6 Technical Changes for Agility

- Traditional systems often include **monolithic applications** with:
  - High **technical debt**
  - Difficult **maintenance and scalability**
- Modern architecture shifts toward:
  - **Microservices**
  - Services developed and deployed **independently**
  - Communication using protocols like **HTTP, TCP, AMQP**

## 3.7 Subscription-Based Economy

### 3.7.1 Market Transformation

- Transition from **product ownership** to **service consumption**
- Economy shifts through stages:
  - **Privately owned** models
  - **Platform economies**
  - **Subscription-based services**
- Services are accessible:
  - On **any device**
  - At **any time**
  - From **any location**



- Payment is made for **access**, not ownership

### 3.7.2 Architectural Demands of Subscriptions

- Subscription services are:
  - **Fluid** (can be paused, modified, resumed, or canceled)
  - Require **scalable, adaptive architectures**
- Systems must support:
  - **On-demand service delivery**
  - **Real-time customization**
  - **Event-driven scaling**

## 3.8 Data as the Foundation of Strategy

### 3.8.1 Importance of Data

- Businesses rely on data to:
  - Understand **client behavior**
  - Track **changing demands**
  - Respond to **dynamic market conditions**
  - Determine **delivery timelines** and **update cycles**
- Challenges include:
  - Data may be **unstructured** or **distributed**
  - Data must be processed securely, respecting:
    - **Confidentiality**
    - **Privacy**
    - **Compliance regulations**

### 3.8.2 Healthcare Industry Example

- Global healthcare under pressure due to:
  - **Aging populations**
  - **Shortage of skilled staff**

- Strategic shift toward **preventive healthcare**
- Focus areas:
  - Encouraging **lifestyle improvements**
  - Developing **personalized health plans**

### 3.8.3 Role of Data

- Collected from:
  - **Medical devices**
  - **Wearables** (e.g., smartwatches)
  - **Gym equipment**
- Used for:
  - **Trend analysis**
  - **Lifestyle interventions**
  - **Commercial opportunities** (e.g., selling health products/services)

### 3.8.4 Challenges in Data-Driven Business Models

- Major challenges:
  - Ensuring **data confidentiality**
  - Achieving **compliance** with international privacy laws (e.g., GDPR, HIPAA)
- Technology for data collection, processing, and analytics is **readily available**
- Greatest difficulty lies in **governance and ethical usage** of data

## 4 Introducing the Scaffold for Multi-Cloud Environment

### 4.0.1 Unplanned Cloud Adoption Pitfalls

- Many businesses begin cloud adoption **without a formal plan**
- Initial steps often involve:

- **Purchasing a subscription**
- **Deploying resources ad hoc**
- Works temporarily for **small-scale environments**, but leads to:
  - **Uncontrolled cost growth**
  - **Lack of visibility** over roles and responsibilities
  - **Security vulnerabilities**
- Analogy:
  - Building a datacenter requires planning and infrastructure
  - Cloud environments require **equally disciplined planning**

#### 4.0.2 Need for Cloud Governance and Structure

- Key requirements before cloud adoption:
  - **Cost visibility and control**
  - **Defined ownership and accountability**
  - **Security of data and digital assets**
  - **Operational clarity** (who manages what and when)
- Planning ensures:
  - **Avoidance of chaos**
  - **Minimization of risks**
  - **Alignment with business goals**

#### 4.0.3 Cloud Adoption Frameworks

- Major cloud providers have introduced **Cloud Adoption Frameworks** (CAFs) to guide:
  - **Strategy creation**
  - **Cloud migration planning**
  - **Governance and security implementation**
- Frameworks help:
  - **Maintain control** during cloud scaling

- **Standardize** cloud environments
- **Align technology** with business and operational requirements
- Frameworks may differ in terminology or structure but share **common foundational principles**

#### 4.0.4 Understanding 'Scaffold' in the Cloud Context

- **Scaffold** (as adopted by Microsoft):
  - A **structural foundation** for deploying and maintaining cloud environments
  - Originates from construction terminology (temporary structure for support)
  - In cloud computing, refers to a **permanent architectural structure**
- Scaffold represents:
  - **Cloud governance foundation**
  - **Support system** for cloud workloads
  - **Blueprint** for building secure and manageable cloud environments

#### 4.0.5 Purpose of Cloud Scaffolding

- Acts as a **landing zone structure**:
  - Designed to manage **multi-service, multi-region deployments**
  - Ensures **security, compliance, and operational readiness**
- Scaffold implementation enables:
  - Structured **cloud onboarding**
  - Defined **network, identity, and access layers**
  - Scalable **infrastructure planning**

### 4.1 Working with Well-Architected Frameworks

#### 4.1.1 Purpose and Scope

- Well-Architected Frameworks guide customers in building **optimized, secure, cost-effective, and reliable** environments in public clouds

- Each cloud provider has a **distinct framework**, but the **core principles are consistent**
- Frameworks support multi-cloud adoption by offering a **standardized approach** despite differences in tools, workflows, service names, and command structures

### 4.1.2 Core Pillars of Well-Architected Frameworks

Most frameworks include the following five core pillars:

1. **Operational Excellence**
  2. **Security**
  3. **Cost Optimization**
  4. **Reliability**
  5. **Performance Efficiency**
- **Sustainability** is an emerging sixth pillar, **currently formalized in AWS only**

### 4.1.3 Operational Excellence

- Focuses on **running workloads efficiently** and **optimizing operational processes**
- Encourages:
  - Use of **automation**
  - Implementation of **Infrastructure as Code (IaC)**
  - Creation of **repeatable, consistent operations**
- Helps design cloud-native operations for:
  - **VMs**
  - **Containers**
  - **Serverless** architectures

### 4.1.4 Security

- Ensures workloads are protected through:
  - **Identity and Access Management (IAM)**
  - **Data protection tools**

- **Security posture management**
- Highlights shared responsibility:
  - **Cloud provider** secures the infrastructure
  - **Customer** secures the applications and data
- Guides compliance with major frameworks:
  - **PCI-DSS**
  - **NIST**
  - **ISO 27001/27002**

#### 4.1.5 Cost Optimization (FinOps)

- Aims to **control and reduce cloud spending**
- Encourages visibility into:
  - **Cloud resource consumption**
  - **Business value derived from expenses**
- Tools and practices include:
  - **Budgeting and alerts**
  - **Rightsizing** workloads (e.g., using serverless instead of VMs)
  - **Reserved Instances** (up to 70% savings over on-demand)

#### 4.1.6 Reliability

- Focuses on ensuring **high availability** and **resilience** of workloads
- Techniques include:
  - **Redundant deployment** across zones and regions
  - **Auto-scaling** to handle peak demand
  - **Automated failovers and recovery mechanisms**
- Supports continuous availability even under **unexpected failures**

#### 4.1.7 Performance Efficiency

- Ensures cloud resources are used **optimally for performance**
- Encourages:

- Use of **scalable architectures**
- Periodic evaluation of **resource types and sizes**
- Leveraging **cloud-native services** that dynamically scale

#### 4.1.8 Sustainability (*included by AWS*)

- Encourages use of cloud infrastructure with:
  - **Lower energy consumption**
  - **Optimized power and cooling usage**
- Focuses on:
  - **Resource efficiency**
  - **Carbon footprint reduction**
  - **Sustainable software design**

#### 4.1.9 Implementation Use Cases

- Frameworks support implementation of:
  - **FinOps**: Financial control of cloud costs
  - **SecOps**: Security governance and compliance
  - **BaseOps**: Operational management and automation
- Serve as **validation checklists** for cloud architects and developers

### 4.2 IAM

- Central concern in IAM:  
Who may do what, when, and why?
- This question governs **authentication** and **authorization** in cloud environments

#### 4.2.1 Everything is an Identity

- In cloud environments, **identity** is not limited to people
- Examples of cloud identities:
  - **Users**

- **Resources**
- **Functions**
- **APIs**
- **Machines**
- **Workloads**
- **Databases**
- All identities must be:
  - **Uniquely identified** (for authentication)
  - **Bound to access rules** (for authorization)

## 4.2.2 Authentication and Authorization

- **Authentication:** Verifying identity (e.g., user login, API key validation)
- **Authorization:** Defining what authenticated identities can do
- Authorization is controlled through:
  - **Access policies**
  - **Roles and permissions**
  - **Scopes and conditions**

## 4.2.3 Identity Directory Systems

- Identity and credential stores play a critical role in IAM
- Common directory systems:
  - **Active Directory**
  - **OpenLDAP**
- Deployment decisions:
  - Host **identity store** within the **public cloud**
  - Or integrate **external IAM systems** that communicate with cloud environments

## 4.2.4 Identity as a Service (IDaaS)

- IAM can be consumed as a **cloud-based service**



- Example solution:
  - **Okta** — provides centralized IAM across **multiple cloud providers**
- Benefits of IDaaS:
  - **Scalable management** of identities
  - **Cross-cloud integration**
  - **Centralized policies and controls**

#### 4.2.5 IAM and Multi-Cloud Architecture

- IAM is the **foundation of governance** in multi-cloud environments
- Consistent IAM practices are essential for:
  - **Access control across providers**
  - **Compliance enforcement**
  - **Minimizing privilege misconfiguration**

### 4.3 Security

#### 4.3.1 Security in Hyperscaler Platforms

- Platforms such as **Azure**, **AWS**, and **GCP** are among the **most secure platforms globally**
- Security is a **shared responsibility model**:
  - **Cloud provider** secures the infrastructure
  - **Customer** secures data, applications, configurations, and access

#### 4.3.2 Security Starts with Policy

- Effective cloud security begins with **organizational policies**
- Businesses must align with:
  - **Regulatory standards**
  - **Industry-specific compliance frameworks**
  - **General security baselines**

#### 4.3.3 Security Frameworks and Standards

- Organizations typically follow **security frameworks** to structure controls and governance

#### 4.3.4 Center for Internet Security (CIS)

- **CIS Baseline:**
  - Offers a **comprehensive set of best practices** for cloud resource hardening
  - Includes:
    - **Scored items:** High-priority controls; must be implemented to avoid audit failures
    - **Non-scored items:** Recommended for stronger security posture but not always enforced
- Auditors focus on **scored items** for compliance assessments

#### 4.3.5 MITRE ATT&CK Framework

- **MITRE ATT&CK** = Adversarial Tactics, Techniques, and Common Knowledge
- Provides a **real-world threat model** based on actual observed cyberattacks
- Tracks:
  - **Attack vectors**
  - **Techniques**
  - **Mitigation practices**
  - Across:
    - **Public cloud platforms** (Azure, AWS, GCP)
    - **Container orchestration platforms** (e.g., Kubernetes)
- Enables security teams to:
  - Understand **current threat landscapes**
  - Design security strategies based on **real attack behaviors**

#### 4.3.6 Cloud Provider Security Tools

- All major cloud platforms provide:

- **Built-in security tools**
- **Monitoring and alerting systems**
- **Role-based access controls (RBAC)**
- **Encryption services**
- **Identity and access governance systems**
- Utilization of these tools depends on **customer configuration and policy decisions**

## 4.4 Cost Management

### 4.4.1 Public Cloud vs. On-Premises Cost Perception

- Common concern:  
"Public cloud is more expensive than traditional on-premises systems"
- This can be true, especially when performing **lift-and-shift** migrations without optimization

### 4.4.2 Reasons for Higher Cloud Costs in Some Cases

#### 1. Incomplete On-Premises Cost Calculations

- Cost comparisons often exclude:
  - **Power and cooling infrastructure**
  - **Physical security and maintenance**
  - **Labor costs** (e.g., system administrators, support staff)
  - **Software licensing**

#### 2. Inefficient Use of Cloud Features

- Public cloud misused as a **static infrastructure**, mimicking on-premises deployment
- Key inefficiencies:
  - Running **workloads 24/7/365** when not needed
  - Ignoring **autoscaling** and **on-demand provisioning**
  - Underutilizing the **pay-as-you-go** model

### 4.4.3 Cost Management Objectives

- Achieve **visibility and control** over cloud expenses
- Ensure spending aligns with:
  - **Business goals**
  - **Authorized usage**
  - **Budget planning**
- Essential integration with:
  - **Identity and Access Management (IAM)**

### 4.4.4 Uncontrolled Cloud Usage Risks

- Potential issues:
  - Provisioning of **expensive services** (e.g., high-performance storage, data lakes)
  - Unauthorized deployment of **cost-heavy resources**

### 4.4.5 Two critical questions:

1. **Was the user authorized** to provision those resources?
2. **Was there a predefined budget** for that deployment?

### 4.4.6 Tracking and Accountability Mechanisms

- Key practices for cost tracking:
  - **Tagging:**
    - Add metadata to resources
    - Examples: owner, department, project, cost-center
  - **Naming conventions:**
    - Enforce predictable, meaningful naming patterns
    - Enable **resource identification**, reporting, and automation
- Support **chargeback and show-back models:**
  - Allocate costs to departments or business units based on usage

## 4.5 Monitoring

### 4.5.1 Need for Visibility and Observability

- Monitoring provides answers to critical operational questions:
  - **What is happening in the environment?**
  - **Why is it happening?**
  - **Is the system healthy in terms of performance and security?**
- Monitoring supports:
  - **Performance tracking**
  - **Security posture awareness**
  - **Issue detection and resolution**

### 4.5.2 Native Monitoring Tools by Cloud Providers

- Major public clouds provide built-in monitoring platforms:

| Cloud Provider      | Monitoring Tool                                              |
|---------------------|--------------------------------------------------------------|
| Microsoft Azure     | <b>Azure Monitor</b>                                         |
| Amazon Web Services | <b>AWS CloudWatch</b>                                        |
| Google Cloud        | <b>Google Cloud Monitoring</b> <i>(formerly Stackdriver)</i> |
| Alibaba Cloud       | <b>CloudMonitor</b>                                          |

- Common functionalities:
  - **Data collection** via monitoring agents (metrics and logs)
  - **Storage** in centralized **log analytics spaces**
  - **Alert policies** for proactive notifications
  - **Dashboards** for visualizing trends and system health

### 4.5.3 Integration with IT Service Management (ITSM)

- Monitoring platforms integrate with external **ITSM systems**, such as:
  - **ServiceNow**
  - Other enterprise helpdesk or incident management tools
- Integration provides:

- **Centralized incident visibility**
- **Unified dashboard view** across platforms
- **Improved workflow and escalation handling**

#### 4.5.4 Third-Party and Alternative Monitoring Solutions

- Common alternatives to native tools:
  - **Splunk**
  - **Datadog**
  - **New Relic**
  - **Prometheus + Grafana (for open-source ecosystems)**
- Benefits:
  - **Cross-platform compatibility**
  - **Advanced analytics capabilities**
  - **Custom dashboards and alerting rules**

#### 4.5.5 Emerging Trend: AIOps (Artificial Intelligence for IT Operations)

- AIOps platforms go **beyond basic monitoring**:
  - Conduct **intelligent, automated analyses** on metrics and logs
  - Provide **predictive insights** and **root-cause analysis**
  - Enable **self-healing systems**
- AIOps relevance:
  - Highly suited for **complex multi-cloud environments**
  - Facilitates **autonomous operations** and **real-time anomaly detection**

### 4.6 Automation

#### 4.6.1 Cost of Labor in IT

- **Labor cost** is the most expensive factor in IT operations.
- Initial outsourcing and offshoring trends were driven by labor cost reduction.

- Enterprises, such as banks, that outsourced IT in earlier decades have **brought IT back in-house** as IT is now a **core business function**.
- Despite in-house operations, the goal of **cost reduction remains a major driver**.

#### 4.6.2 Automation as a Cost Optimization Strategy

- **Automation helps reduce the dependency on manual labor**, thereby lowering costs.
- Key concepts enabling automation:
  - **Infrastructure as Code (IaC)**
  - **Configuration as Code (CaC)**
  - **Code repositories** for versioning and collaboration
  - **DevOps pipelines** for continuous integration and deployment (CI/CD)
- Automation ensures **repeatable, consistent deployments** with minimal manual intervention.

#### 4.6.3 Advantages of Automation

- **Minimizes human error:**
  - Major IT outages often caused by manual mistakes.
  - Automation improves **consistency and reliability**.
- **Speeds up deployment and updates.**
- **Improves scalability** of environments.
- **Enables agility** for modern businesses operating in competitive markets.

#### 4.6.4 Limitations and Considerations

- Automation is **not inherently fault-tolerant** unless thoroughly tested.
- Still requires:
  - **Skilled developers** to write automation scripts and IaC modules.
  - **Robust testing and validation** mechanisms before deployment.
- Automation **should be seen as a process discipline**, not just a technology choice.

#### 4.6.5 Process-Centric View of Automation

- Modern automation aligns with **process methodologies**, not just tool usage.
- Example methodology:
  - **Site Reliability Engineering (SRE)** developed by Google.
    - Combines **software engineering** and **IT operations**
    - Focuses on **service reliability** through automation, monitoring, and incident response

## 5 Understanding Identities and Roles in the Cloud

### 5.1 Everything in the Cloud Has an Identity

- Every **person, resource, service, function, API, VM**, and **database** in the cloud is treated as an **identity**.
- Two core operations associated with identity:
  - **Authentication** – Verifying the identity.
  - **Authorization** – Granting access rights based on identity.

### 5.2 Identity Store and Authentication Mechanism

- Most enterprises use **Active Directory (AD)** as the central identity store.
- AD stores identities of:
  - **Persons** (e.g., users, admins)
  - **Machines** or **computers** (e.g., VMs)
- AD functions using **domains**, and resources must be **domain-joined** for meaningful integration.
  - **Domain-joining** allows cloud resources to be governed by enterprise policies.

### 5.3 Extending Domains to Public Cloud Platforms

- Public cloud domains:



- Azure: .onmicrosoft.com
- AWS: .aws.amazon.com
- GCP: .cloud.google.com
- Enterprises must extend their own domain to the cloud using **domain controllers** to enable domain federation.
  - Example: myfavdogbiscuit.onmicrosoft.com on Azure

## 5.4 AD Federation and Integration

- **Active Directory Federation** connects enterprise identity stores to cloud domains.
- AD uses **Kerberos** for authentication.
- All major cloud platforms support:
  - **Kerberos**
  - **LDAP (Lightweight Directory Access Protocol)**
  - **Active Directory Federation**

## 5.5 External Identities and Cross-Domain Access

- Granting access to external identities (not part of the enterprise domain):
  - Azure: **Azure AD B2B**
  - AWS: **Amazon Cognito**
  - GCP: **Cloud Identity**

## 5.6 Authorization and Role-Based Access

- After authentication, authorization is defined using **Role-Based Access Control (RBAC)**:
  - Azure: **RBAC**
  - AWS: **IAM**
  - GCP: **Cloud IAM / Cloud Identity**
- Access is based on **roles**, which define:
  - **Who** can do **what**

- **When and under what conditions**
- **Groups** of identities can be managed together using the same RBAC policies.

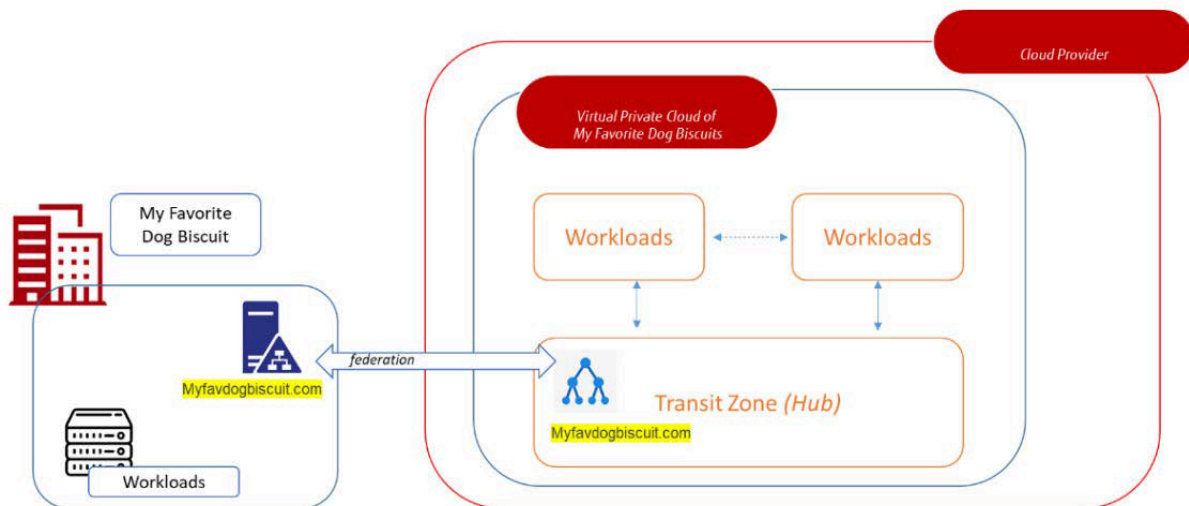


Figure - 4 - Active Directory Federation

## 5.7 Multi-Factor Authentication and Compute Resource Authentication

- For human users:
  - **Multi-Factor Authentication (MFA)** is recommended for secure login.
- For system resources:
  - Authentication is typically done using **key-based access** (e.g., complex hash-based tokens).

## 5.8 Azure AD vs. On-Premise AD

- **Active Directory (AD):**
  - Traditional directory service
  - Uses **Kerberos** for authentication
- **Azure Active Directory (Azure AD):**
  - Native identity provider in Azure
  - Designed for **cloud-native identities**

- Does **not use Kerberos** by default
- Azure AD federates with on-prem AD for enterprise-wide authentication.

## 6 Defining Architecture Principles for Multi-Cloud

### 6.1 Foundation: Architecture Development Method (ADM)

- **ADM (Architecture Development Method)** is a widely accepted framework under **TOGAF**.
- Used to structure enterprise architecture planning and execution.
- Follows a **cyclical model**, starting with preliminary groundwork and leading into architecture phases.

### 6.2 Preliminary Phase

- First step before defining business architecture.
- Establishes:
  - **Architecture principles**
  - **Reference models**
  - **Frameworks to be used**
- Output of this phase feeds directly into the **Architecture Vision** phase.

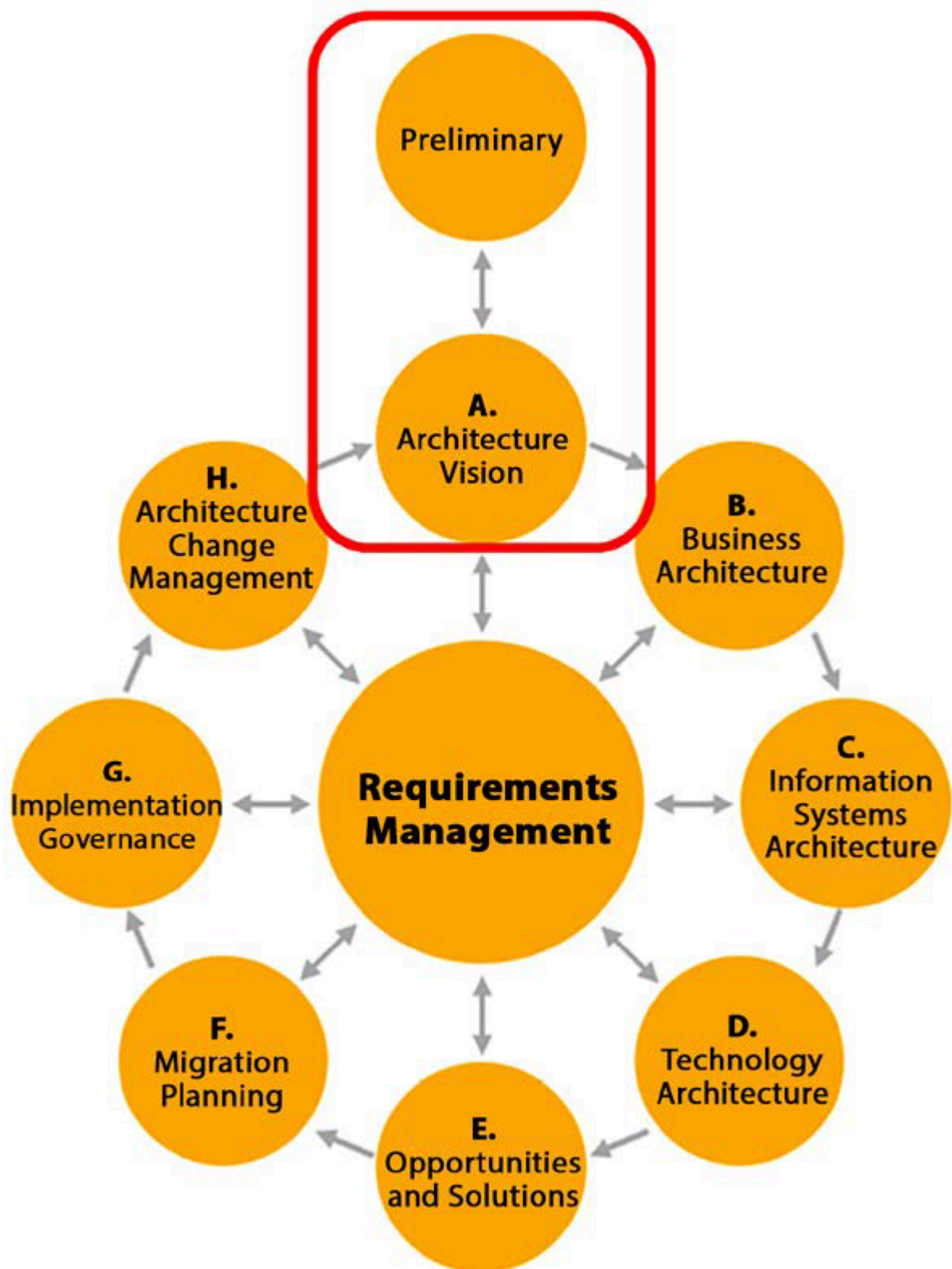


Figure - 5 - The preliminary phase and architecture vision in TOGAF's ADM cycle

## 6.3 Importance of Architecture Principles

- **Architecture principles** guide all architecture decisions.

- Provide **stable, long-term**, and **consistent** rules.
- Must **align with business goals**.

## 6.4 Key Criteria for Good Architecture Principles

1. **Business-aligned:** Principle must directly support enterprise goals.
2. **Unambiguous:** Principle should be clearly defined and not open to multiple interpretations.
3. **Actionable:** Principle must guide solution design and implementation clearly.

## 6.5 Clarifying Misconceptions

- "Cloud-first" or "Moving to the cloud" are **technology choices**, not business goals.
- Architecture principles must not be confused with tool-based strategies or temporary trends.

## 6.6 Common Architectural Principles When Moving to Cloud

- **Flexibility:** Capability to scale or shift workloads and services as per demand.
- **Agility:** Ability to respond to change rapidly and deploy features faster.
- **Cost-efficiency:** Optimizing cost without compromising functionality or performance.
  - Requires accurate calculation of Total Cost of Ownership (TCO) and Return on Investment (ROI).
  - Misinterpretation of cost-efficiency can lead to poor cloud decisions.

## 6.7 Suggested Categories for Grouping Architecture Principles

| Category                   | Focus Area                                          |
|----------------------------|-----------------------------------------------------|
| <b>Business Principles</b> | Strategic alignment, value creation, time-to-market |

|                                        |                                                                |
|----------------------------------------|----------------------------------------------------------------|
| <b>Security &amp; Compliance</b>       | Risk mitigation, regulatory adherence, access control          |
| <b>Data Principles</b>                 | Accuracy, privacy, ownership, systems of record                |
| <b>Application Principles</b>          | Modularity, portability, standardization, cloud-native support |
| <b>Infrastructure &amp; Technology</b> | Scalability, availability, fault tolerance, automation         |
| <b>Usability Principles</b>            | End-user experience, accessibility, device compatibility       |
| <b>Process Principles</b>              | Automation, agility, DevOps enablement, lifecycle integration  |

## 6.8 Architectural Decision Validation

- Each principle should help validate and justify architectural choices.
- Principles become the **lens** through which trade-offs are evaluated during design.
- They are not absolute rules, but strong guidelines that shape the entire architecture vision.

# 7 Using Quality Attributes in Architecture

## 7.1 Operability

- Focuses on ensuring systems remain operational, secure, and observable.
- Includes:
  - **Automation** of deployment and operations.
  - **Monitoring and Logging:**
    - Must define what to monitor and to what extent.
    - Emphasis on **end-to-end monitoring** across multi-cloud components.
    - Monitoring relates not just to system health, but also to correct functioning and stability.
  - **Log Management:**

- Logs must be stored appropriately and retained as per audit requirements.
- **Performance Monitoring:**
  - Directly tied to user experience.
  - Requires minimal resource consumption from agents used in monitoring.

## 7.2 Performance

- Defined from an **end-user perspective**.
- Requires:
  - **End-to-end visibility** from interface to backend infrastructure.
  - Use of **heartbeat signals** and transaction tracing to measure health and speed.
  - Awareness of **agent footprint** (CPU/memory usage) to avoid performance degradation.

## 7.3 Configurability

- Systems must adapt quickly to changes.
- Requires setting and managing **desired state configurations** for resources.
- Configuration examples:
  - CPU, memory, and storage settings.
  - Boot sequence and security settings.
  - Access control and OS parameters.
- Automation using tools such as:
  - **PowerShell DSC, AWS Systems Manager, GCP Compute Engine.**

## 7.4 Discoverability

- Ensures resources are visible and can communicate with other components.
- Includes **service discovery**:

- Critical for **microservices** to dynamically locate each other.
- Uses central service registries.
- Also important for:
  - Monitoring integration.
  - Operational efficiency in dynamic environments.

## 7.5 Security

- Primary objective: **Protect data integrity**.
- Must be approached by:
  - Identifying threats and potential breaches.
  - Implementing security policies, monitoring, and alert response.
- Involves:
  - Architecture alignment with **SecOps** processes.
  - Data-level and app-level protection strategies.

## 7.6 Scalability

- A core feature of public cloud platforms.
- Includes:
  - **Horizontal scaling** (scaling out): Adding more VMs or instances.
  - **Vertical scaling** (scaling up): Adding CPU/memory to a resource.
  - **Downscaling**: Reducing resource usage to cut costs.
- Key Concepts:
  - **Scale units**: VMs, disks, databases, storage accounts.
  - **Autoscaling**:
    - Trigger-based scaling automation.
    - Cost thresholds and impact awareness required.
  - **Partitioning**:
    - Separating data/apps to reduce contention and improve scalability control.



## 7.7 Robustness (Availability and Recovery)

- Goal: Ensure business continuity through resilient architecture.
- Must cover:
  - **Compute, application, and data** layers together.
- Includes:
  - **Recovery Objectives:**
    - **RPO (Recovery Point Objective):** Acceptable data loss window.
    - **RTO (Recovery Time Objective):** Acceptable downtime duration.
  - **Backup and Restore Strategies:**
    - Use snapshots and incremental backups.
    - Regular testing of restore procedures is essential.
  - **Availability Design:**
    - Align infrastructure availability to application needs (e.g., 99.9% app uptime).
    - Consider global availability and possible outages in public cloud.

## 7.8 Portability

- Enables applications and data to move across platforms.
- Reduces **vendor lock-in** and supports flexibility.
- Requirements:
  - Abstraction between app logic and system interfaces.
  - Use of **containerization** (e.g., Kubernetes).
- Challenges:
  - Platform-specific native technologies.
  - **Data gravity:** Massive datasets hinder movement between platforms.

## 7.9 Usability

- Relates to **ease of access** and **application interaction**.

- Architectural implications:
  - Transparent multi-cloud user experience.
  - Routing and connectivity design for internet/office access.
  - May require:
    - **DMZ configurations.**
    - **Jump boxes** in multi-cloud network architecture.
- Principle of **technology transparency**:
  - Users should not be exposed to backend complexity.
  - Architectures should deliver secure and seamless user interaction.

## 7.10 Evolving Frameworks for Agile Architecture

- **Continuous Architecture**: Emphasizes adaptability to change and ongoing delivery.
- **Open Agile Architecture (O-AA)**:
  - Created by The Open Group.
  - Prioritizes **customer experience** and **business agility**.
  - Considers evolving user interaction across digital channels (web, mobile, IoT, etc.).

# 8 Defining Principles from Use-Cases

## 8.0 Business Use Case

- Describes actions required to deliver a product/service to the customer.
- Includes:
  - Defined sequence of steps
  - Customer value representation (price)
  - Cost–margin relationship for profitability
- Guides the derivation of architecture principles.

## 8.1 Business Principles

- Derived from business goals and strategies aligned with mission statements.
- Goals may include:
  - Faster customer response
  - Quicker product launches (time to market)
  - Improved product/service quality
  - Enhanced employee engagement
  - Launch of digital platforms (websites/webshops)
  - Increased revenue and controlled costs
- Goals must be SMART: Specific, Measurable, Attainable, Relevant, Timely.
  - Example: Launch of product X webshop in North America by June 1.

## 8.2 Security and Compliance Principles

### 8.2.1 Zero Trust

- Nothing is trusted by default inside the network.
- Every entity (device, app, user) is verified.
- Platform micro-segmentation is enforced.
- Treats every system as already breached (based on frameworks like MITRE ATT&CK).

### 8.2.2 Security by Design

- Security embedded in architectural components.
- Systems are hardened; code is secured using encryption or hashing.
- Security frameworks such as CIS (Center for Internet Security) are applied.
  - Example: CIS Control 13 advises encryption for data in transit and at rest.

### 8.2.3 Compliance

- Ensures adherence to regional/national/global laws and regulations.

- Example: GDPR compliance
- Leads to architectural requirements like:
  - Database security
  - Data encryption
  - Access control using authentication and authorization
- Compliance complexity increases in multi-cloud due to diverse providers.

## 8.3 Data Principles

- Key data principles:
  - **Accuracy:** Data must be correct and reliable.
  - **Relevance:** Data should be contextually appropriate.
  - **Timeliness:** Data must be current.
  - **Consistency:** Standardized formats and units.
  - **Privacy and Security:** Protect sensitive data from unauthorized access.

### 8.3.1 Data System Types

- **Systems of Record:**
  - Store structured/raw data (e.g., ERP, CRM, CMS).
  - Supported by technologies like data lakes and multi-DB stores.
- **Systems of Engagement:**
  - Interact with or collect data (e.g., mobile apps, IoT, emails).
- **Systems of Intelligence** (emerging):
  - Analyze data using AI/ML to drive automated business decisions.
  - Enabled by tools like Azure ML, AWS SageMaker, GCP Cloud AI.

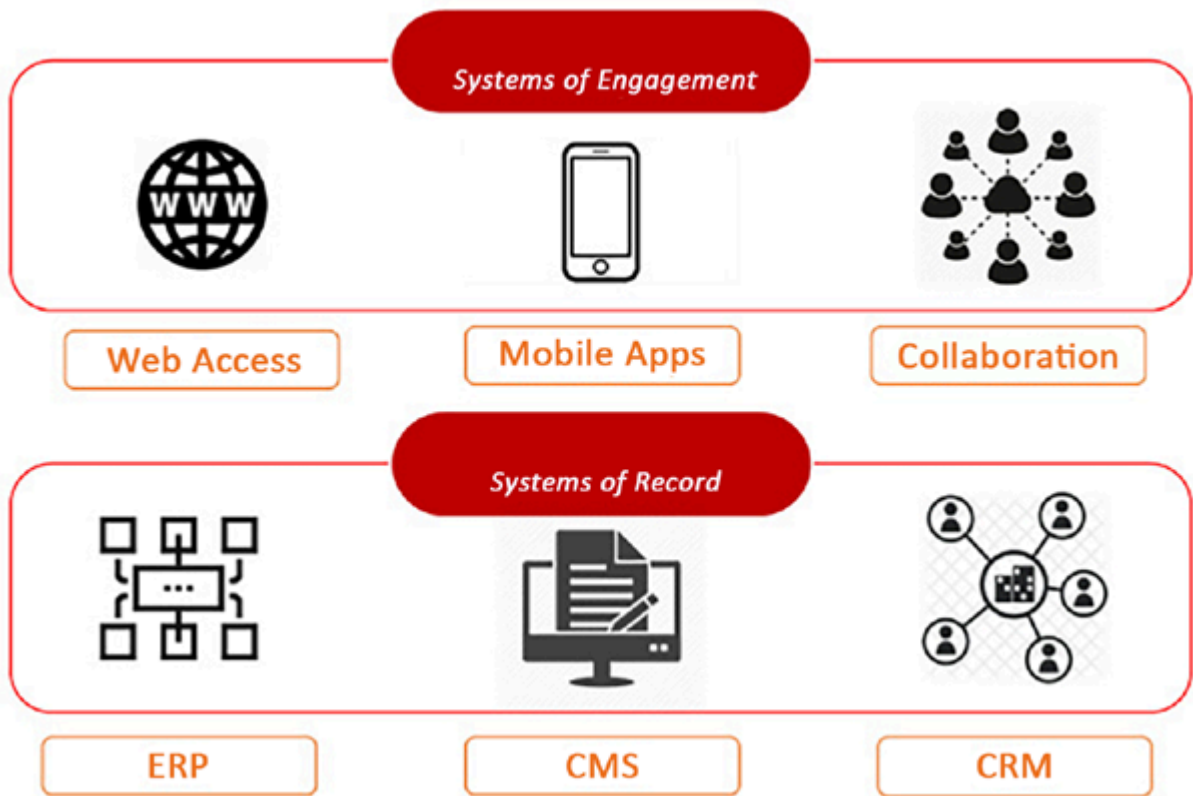


Figure - 6 - Simple representation of systems of engagement and systems of record

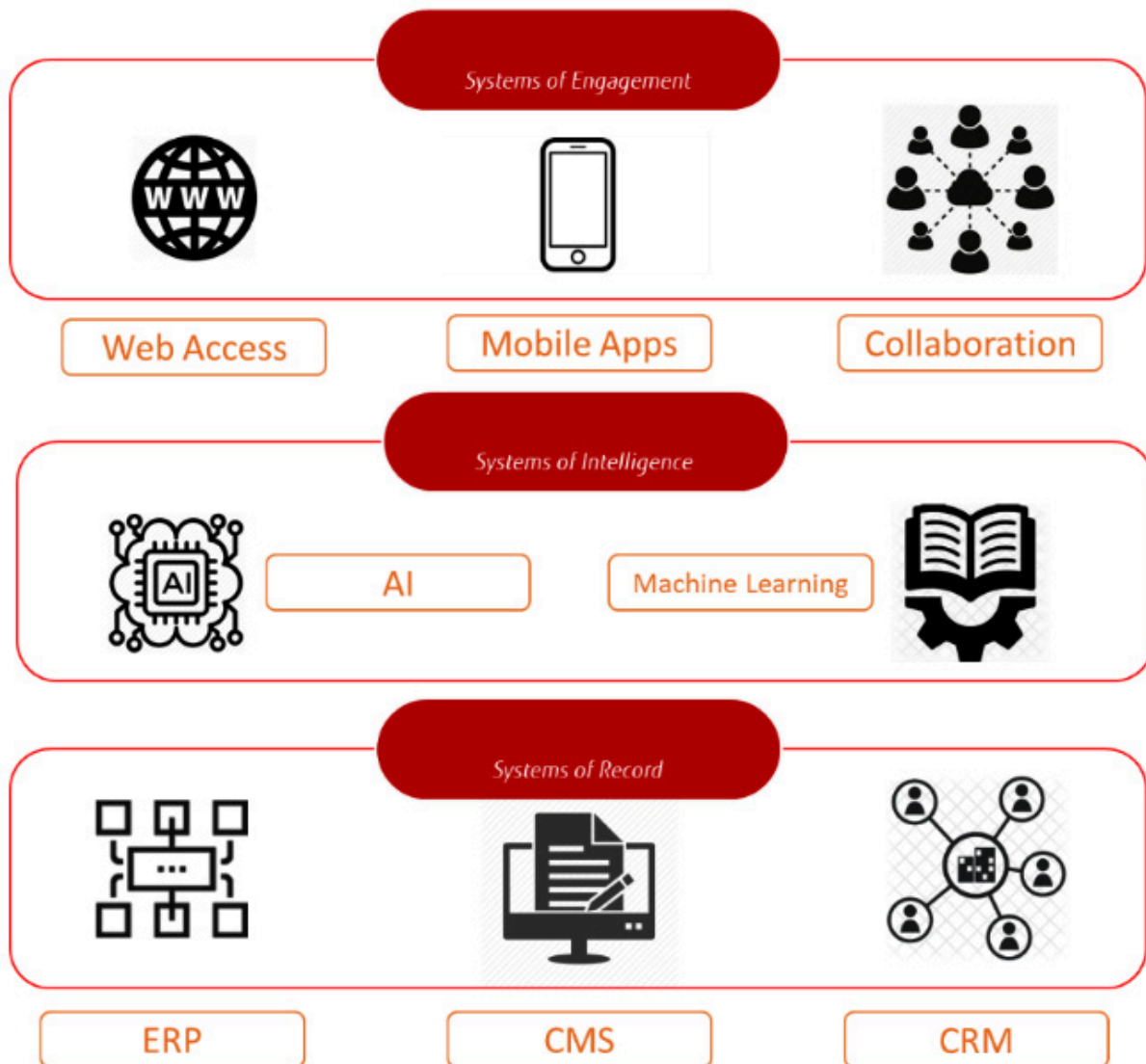


Figure - 7 - Simple representation of the systems of intelligence layer

## 8.4 Application Principles

- Applications must enable:
  - Data accessibility
  - Data usability
  - Maintenance of data quality
- Based on data-driven architecture principles.
- In multi-cloud:
  - Cloud-native applications replace monolithic systems.
  - Use microservices, APIs, triggers, containerized platforms.

- Key characteristics of modern applications:
  - Platform independence
  - Scalability
  - Open standards and interoperability
  - Support for mobility
  - Agile development capability
  - Well-documented code and runbooks

## 8.5 Infrastructure and Technology Principles

- Focus on platform, topology, interdependencies, and landing zones.
- Principles should be **guiding**, not **product-specific**.
- Key principles:
  - Scalability (horizontal and vertical)
  - Support for micro-segmentation
  - Adherence to the Twelve-Factor App model:
    - Platform portability
    - Minimal difference between development and production environments
    - Scalable architecture

## 8.6 Process Principles

- Emphasize **automation** and **workflow definition**.
- Encourage code-based operations:
  - All deployments from source code (e.g., master branch)
  - Testing required before code merges
- Align with **Agile** and **DevOps** culture:
  - Frameworks: SAFe, Spotify model
  - Principles define ways of working, not implementation details